- A new root node is created. It holds data item 1 of the node being split and becomes the parent of the old root node.
- A second new node is created. It becomes a sibling of the node being split.
- Data item 2 of the split root is moved into the new sibling.
- The old root maintains data item 0 (discarding its references to data items 1 and 2).
- The two rightmost children of the old root node being split are disconnected from it and connected to the new sibling node.

Figure 9-5 shows the root being split. This process creates a new root that's at a higher level than the old one. Thus, the overall height of the tree is increased by one. Another way to describe splitting the root is to say that a 4-node is split into three 2-nodes.

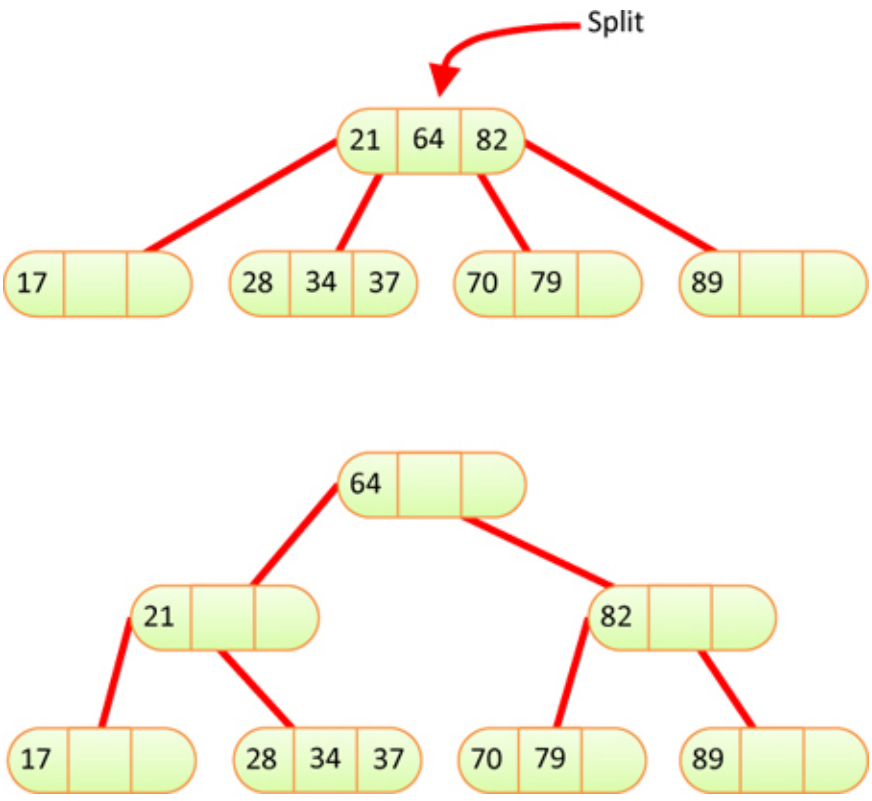

Figure 9-5 *Splitting the root*

As mentioned before, the node splits happen during the search for an insertion node. After each split, the search for the insertion point continues down the tree, possibly finding more full nodes that need to be split. In Figure 9-5, if the data item to be inserted had a key between 21 and 64, the search for the insertion node would lead to the node containing 28-34-37. That node would have to be split because it is full. Inserting in any other leaf node would not require another split.

Splitting on the Way Down

Notice that, because all full nodes are split on the way down, a split can't cause an effect that ripples back up through the tree. The parent of any node that's being split is guaranteed not to be full and can therefore accept data item 1 without itself needing to be split. Of course, if this parent already had two children when its child was split, it will become full with the addition of data item 1. Becoming full means that it will split when the next insertion search encounters it.

Figure 9-6 shows a series of insertions starting with an empty tree and building up to a three-level tree. There are five node splits: two of the root and three of leaves. After 79 is inserted, the tree is the same as in the top of Figure 9-5. Unlike that example, the next insertion of 49 in Figure 9-6 causes two splits: the one at the root and the one at the leaf node containing 28-34-37. Splitting the root adds the third level, and splitting the leaf node enables the insertion of 49 after 37. The tree remains balanced at every stage, and the number of levels increases whenever the root node is split.

Figure 9-6 *Insertions into a 2-3-4 tree*

The Tree234 Visualization Tool

Operating the Tree234 Visualization tool provides a way to see the details of how 2-3-4 trees work. When you start the visualization tool, you see a screen like Figure 9-7.

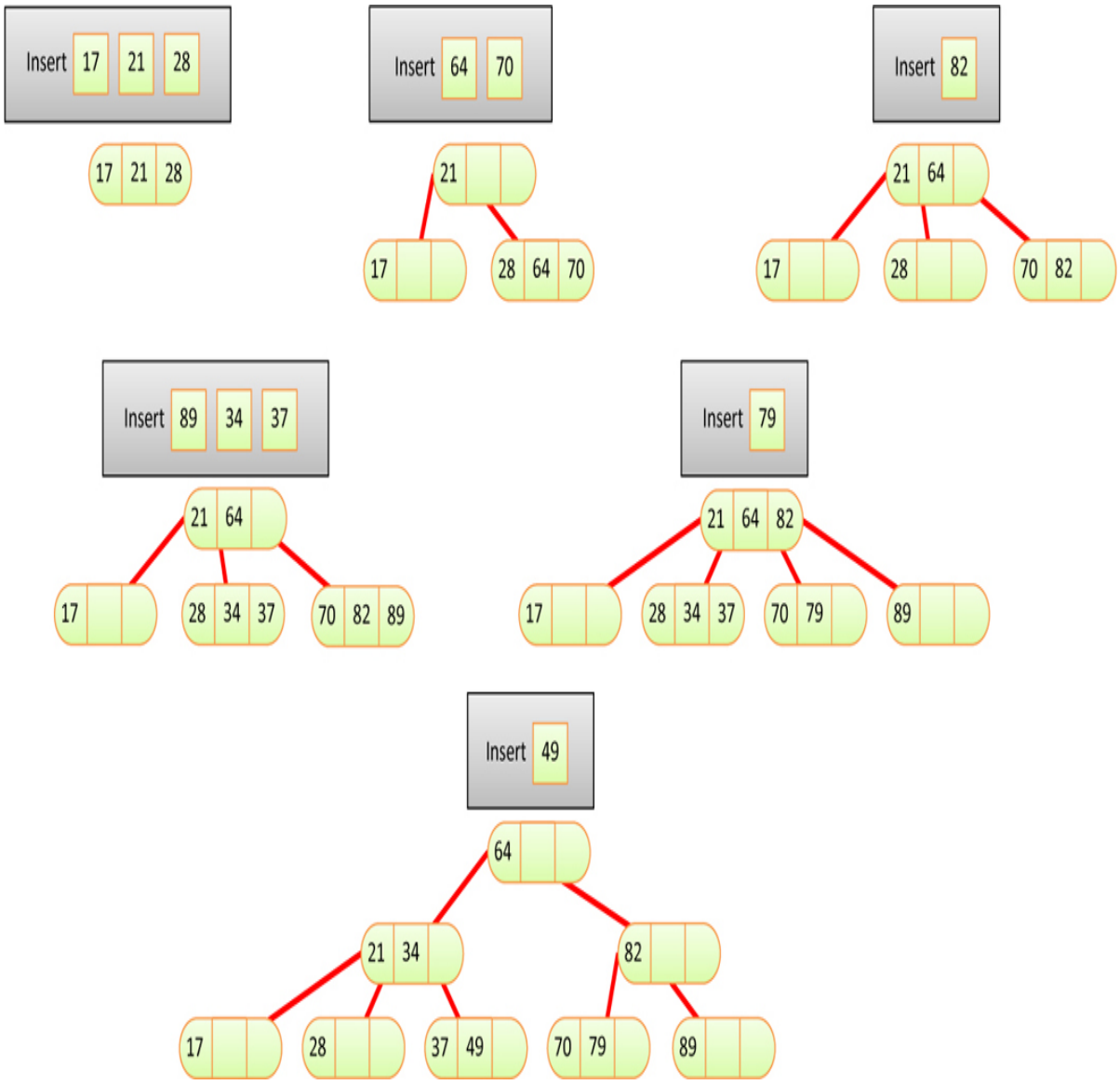

Figure 9-7 *The Tree234 Visualization tool*

The tree is initially empty when you start the tool. The `Tree234` label at the top is for the tree object itself, with an empty pointer to the root node. This matches the structure you saw for binary search trees in Chapter 8.

The Random Fill and New Tree Buttons

You can use the Random Fill button to add items to the tree. You type the number of data items to add in the text entry box and select Random Fill. Try adding a few items (up to three) to an empty tree to see the creation of the first node. As with the binary search tree, each item has a numeric key and a colored background to represent the data stored with the key.

Filling in many items (the maximum allowed is 99) will create a large 2-3-4 tree. Because 2-3-4 trees can be very "bushy" (wide), they are difficult to fit easily on the screen. The visualization provides tools for zooming and scrolling over the larger trees, but it is easier to learn how 2-3-4 trees operate by focusing on smaller trees. To keep all the nodes visible, fill empty trees with 10 or fewer data items.

If you create a large tree and want to return to smaller ones, use the New Tree button to return to the empty tree. You can then add new items either randomly or using the Insert button that we describe shortly.

The Search Button

You can watch the visualization tool locate a data item by typing its key into the text entry box and selecting the Search button. Clicking an existing data item with your pointer device enters its key in the text entry box. You can use the Step and Pause/Play buttons to stop the animation to observe the individual steps.

A search involves examining one node on each level. Within each nonleaf node, the algorithm examines each data item, starting on the left, to see if it matches the search key or, if not, which child it should go to next. In a leaf node, it examines each data item to see whether it matches the search key. If it can't find the specified item in the leaf node, the search fails.

The Insert Button

The Insert button causes a new data item, with a key specified in the text box, to be inserted in the tree. The algorithm first searches for the appropriate leaf node. If it encounters a full node along the way, it splits that node before continuing. Splitting nodes widens the tree, so parts of the expanded tree may

be placed outside the visible region. Splitting the root node makes the tree grow taller and narrower. For tall trees, the root node may appear far to the right, in anticipation of further splits at level 1 that broaden the tree.

Experiment with the insertion process. Watch what happens when there are no full nodes on the path to the insertion point. This process is straightforward. Then try inserting at the end of a path that includes a full node, either at the root, at the leaf, or somewhere in between. Watch how new nodes are formed and the contents of the node being split are distributed among three different nodes.

Zooming and Scrolling

One of the problems with viewing 2-3-4 trees is that there are a great many nodes and data items just a few levels down. Keeping the number of levels to a minimum benefits the search speed, of course, but makes it hard to see all the items on a particular level when displayed horizontally. Levels 0, 1, and 2 are fairly easy to see, but when the tree grows to level 3 and deeper, the leaf nodes become spread over a long distance.

You can move around the tree using the scrollbars that appear when tree size is larger than what can be displayed in the window. You can also zoom in and out to focus on a particular part of the tree or try to see the overall shape. The Zoom In and Zoom Out buttons let you narrow or broaden the focus around what is in the center of the view.

You also can zoom by double-clicking the tree or its background. Each double-click zooms in a step while keeping the clicked point in the same place on the screen. Holding down the Shift key or using the second mouse button when double-clicking zooms out a step around the clicked point. When one or both scrollbars disappear, centering around the clicked point may no longer be possible. Scroll wheels should also work to change the zoom factor. Returning to an empty (New) tree restores the zoom factor and removes the scrollbars.

For example, if you start with an empty tree and insert 55 random data items, you produce a view something like Figure 9-8 where only 11 of the items are visible.

Figure 9-8 *The zoomed-in view on a tree with 55 data items*

If you zoom out five steps, the view becomes what's shown in Figure 9-9. You can see data items at all four levels now, but the individual items become illegible. The gaps exist to accommodate new nodes at the leaf level.

Figure 9-9 *The zoomed-out view on the same 55 data item tree*

Using the zoom and scrolling controls allows you to see both the big picture and the details and, we hope, put the two together in your mind.

During animated operations, the code keeps pointers to one or two nodes in the tree. The visualization tool attempts to keep those active pointers visible on the screen by changing the scrollbars. The program's manipulation of the scrollbars can lead to big jumps in viewpoint when you're zoomed in to a few nodes of large trees. The tool does not change the zoom factor, leaving that to you. Changing the zoom while the animation is paused is possible if you double-click but may leave some pointers in the wrong locations.

Experiments

The Tree234 Visualization tool offers a quick way to learn about 2-3-4 trees. Try inserting items into the tree. Watch for node splits. Stop before one is about to happen and figure out where the three data items from the split node are going to go. Then resume the animation to see whether you're right. As the tree gets larger, you'll need to move around it to see all the nodes.

How many data items can you insert in the tree? There's a limit because only keys with values 0 to 99 are allowed. Those hundred items could be packed into as few as 34 nodes because each node can hold 3 items.

How many levels can there be? At level 0, there is exactly 1 node. At level 1, there are 4, and at level 2, 16. A maximum-sized tree occupies at least 34 nodes, so it would have to have some nodes at level 3. Depending on the order the items are inserted, however, some splits could cause the tree to reach level 4. Are there cases when it could go to 5? to 6?

You can insert the most items in the fewest levels by deliberately inserting them into nodes that lie on paths with no full nodes, so that no splits are necessary. Of course, there is no way to guarantee such an ordering with real data.

Python Code for a 2-3-4 Tree

In this section we examine a Python program that implements a 2-3-4 tree. We introduce the code in sections. You can get the full `Tree234.py` program in the supplemental files with this text, and you can see most of it in the code window of the visualization tool. The program is relatively complex, although it's similar in some ways to the `BinarySearchTree` that was described in Chapter 8. It's often best to peruse the entire program in an editor to see how it works.

Like the `BinarySearchTree`, this data structure uses two classes: `Tree234` and `__Node`. `Tree234` is the public class, and `__Node` is a private class defined within it. Let's start with the `__Node` class.

The __Node Class

Objects of the `__Node` class represent the individual nodes of the tree. They manage the items stored at the node as well as the references to any child

subtrees. We define the `__Node` class as a private class within the `Tree234` class to keep calling programs from making changes to the tree's internal relationships.

As shown in Listing 9-1, the `Tree234` class defines relevant constants for the maximum number of child links and key-value pairs it can store. The `__Node` class uses these constants in allocating arrays for data. In this implementation, the keys, their corresponding values, and related children are kept in separate arrays. This arrangement contrasts with other data structures that store a single object for each data item and use a key function to extract the key value from the object. Both styles have advantages.

Listing 9-1 *The `__Node` Class Within the `Tree234` Class*

```
class Tree234(object):          # A 2-3-4 multiway tree class

   maxLinks = 4                 # Maximum number of child links
   maxKeys = maxLinks - 1       # Maximum number of key-value pairs

   class __Node(object):        # A node in a 2-3-4 tree
      def __init__(             # Constructor takes a key-data pair
            self,               # since every node must have 1 item
            key,                # It also takes a list of children,
            data,               # either empty or a pair that must
            *children):         # both be of __Node type.
         valid = [x for x in children # Extract valid child links
                  if isinstance(x, type(self))]
         if len(children) not in (0, 2): # Check number of children
            raise ValueError(
               "2-3-4 tree nodes must be created with 0 or 2
children")
         self.nKeys = 1         # Exactly 1 key-data pair is kept
         self.keys = [key] * Tree234.maxKeys # Store array of keys
         self.data = [data] * Tree234.maxKeys # Store array of values
         self.nChild = len(valid) # Store number of valid children
         self.children = (      # Store list of child links
            valid + [None] * (Tree234.maxLinks - len(valid)))

      def __str__(self):        # Represent a node as a string of keys
         return '<Node234 ' + '-'.join( # joined by hyphens w/ prefix
            str(k) for k in self.keys[:self.nKeys]) + '>'
```

```
    def isLeaf(self):       # Test for leaf nodes
        return self.nChild == 0
```

The constructor for `__Node` objects takes a single `key` and `data` value as input. The reason is that nodes in the 2-3-4 must always have at least one item stored in them; there's never an empty node. The node may be a leaf node or an internal node holding one item. Nodes are created when other nodes split (other than the first root). Depending on whether the node being split is the root, an internal node, or a leaf node, the node being constructed will have either two or no children. The asterisk before the `children` parameter of the constructor means that Python will interpret any arguments after the `data` argument as child nodes.

The `children` arguments that are the same type as the `__Node` object are placed in the `valid` list. That filtering is done by using a list comprehension—`[x for x in children if isinstance(x, type(self))]`—which uses `x` as a variable to go through all the `children` items and test that they are instances of this object's type. The first `if` statement checks whether the caller passed an appropriate number of valid child nodes to the constructor. More than two is not allowed because the node will have only one item. A single child is never allowed in 2-3-4 trees. When the number of children isn't zero or two, the constructor raises an exception to report the problem.

After the number and type of children provided are verified, the constructor initializes the fields of the object. The number of key-value pairs, `nKeys`, is set to 1. The `key` and `data` are put into arrays of size `maxKeys`. The constructor allocates the full array to enable shifting keys and values later when more items are added or removed. Note the arrays are initialized to have three copies of the `key` and `data` in them, but the extra copies are ignored because `nKeys` is 1. Similarly, the child links are stored in the `children` array, and the quantity is kept in the `nChild` field. The children array is padded with `None` to ensure `maxLinks` cells are allocated.

We provide a `__str__()` method to enable inspection of nodes and printing trees. This function converts a node containing the keys 27 and 48 to the string `'<Node234 27-48>'` by joining the list of keys with the hyphen character. The keys put in the string are limited by the node's `nKeys` attribute. The `isLeaf()` method checks whether the node is a leaf with no children or an internal node.

After constructing a 2-3-4 node, you need to be able to insert other data items into it. In contrast to binary search trees, 2-3-4 nodes hold more than one key

and datum. Insertion occurs after splitting any full 2-3-4 nodes and finding a leaf node where the new item's key belongs. Insertion can also occur when splitting a node and inserting key 1 in a parent node. The `insertKeyValue()` method shown in Listing 9-2 takes a `key`, a `data` value, and an optional `subtree` to perform this action. The subtree is needed when inserting the item in an internal node during a split.

Listing 9-2 *The `insertKeyValue()` Method of `__Node` Objects*

```
class Tree234(object):          # A 2-3-4 multiway tree class
…
   class __Node(object):        # A node in a binary search tree
…
      def insertKeyValue(       # Insert a key value pair into the
            self,               # sorted list of keys. If key is already
            key,                # in list, its value will be updated.
            data,               # If key is not in list, add new subtree
            subtree=None):      # if provided, just after the new key
         i = 0                  # Start looking at lowest key
         while (i < self.nKeys and # Loop until i points to a key
                self.keys[i] < key): # equal or greater than goal
            i += 1              # Advance to next key
         if i == Tree234.maxKeys: # Check if goal is beyond capacity
            raise Exception(
               'Cannot insert key into full 2-3-4 node')
         if self.keys[i] == key: # If the key is already in keys
            self.data[i] = data # then update value of this key
            return False        # Return flag: no new key added
         j = self.nKeys         # Otherwise point j at highest key
         if j == Tree234.maxKeys: # Before shifting keys,
            raise Exception(  # raise exception if keys are maxed out
               'Cannot insert key into full 2-3-4 node')
         while i < j:           # Loop over keys higher than key i
            self.keys[j] = self.keys[j-1] # and shift keys, values
            self.data[j] = self.data[j-1] # and children to right
            self.children[j+1] = self.children[j]
            j -= 1              # Advance to lower key
         self.keys[i] = key     # Put new key and value in hole created
         self.data[i] = data    # by shifting array contents
         self.nKeys += 1        # Increment number of keys
         if subtree:            # If a subtree was provided, store it
            self.children[i + 1] = subtree # in hole created
            self.nChild += 1    # This node now has one more child
         return True            # Return flag: a new key was added
```

Inserting a key in a sorted array is the same process the insertion sort used in Chapter 3, "Simple Sorting." Because we've chosen to store keys, values, and child trees in separate arrays with a related index, it's a little complex to reuse the `SortArray` class. Instead, we implement a simple `while` loop to find the insertion index of the new `key`. Because the maximum array is only three elements long, we can use a linear search rather than a binary search with little effect on performance. After `i` is set to the index where the `key` should be inserted, the loop exits, and the method raises an exception if the insertion would overflow the array.

The next `if` statement checks whether the `key` to insert duplicates an existing key. In that case, it simply updates the data value associated with that `key` and returns a Boolean flag to indicate that no new key was inserted. Alternatively, the method could raise an exception to disallow duplicate keys.

The next section of `insertKeyValue()` uses index `j` to index the highest key that must be stored in the node. If that value for `j` shows that the node is already full, `j == Tree234.maxKeys`, an exception is raised, instead of overflowing the storage. The `j` variable points at the cell just after the last active key. The `while` loop shifts keys, data values and child links to higher indices to make room for the insertion at index `i`. The references in the `children` array are offset by one from the keys and data arrays. After shifting all the cell values, the new key and data can be inserted at index `i`. The new subtree is also inserted, if provided.

The `insertKeyValue()` method finishes by returning `True` to indicate the insertion added a key and value. You see how that return value benefits the caller for different kinds of 2-3-4 tree inserts in the next section.

The Tree234 Class

An object of the `Tree234` class represents the entire tree. The class has only one field, `root`, which is either a `__Node` object or `None`. All public operations start at the root, but several methods can benefit from having private, recursive methods that operate on subtrees specified by a particular `__Node` object.

The constructor for `Tree234` objects shown in Listing 9-3 simply creates an empty tree by setting the `root` field to `None`. The `isEmpty()` method checks whether any items have been inserted in the tree by comparing `root` with `None`.

The `root()` method uses it to raise an exception on empty trees. For trees containing nodes, it returns arrays of the data and keys that the root node contains. The Python array lengths tell whether the root node contains one, two, or three items.

Searching

Searching for a data item with a specified `goal` key, possibly for insertion, is carried out by the `__find()` routine. The one shown in Listing 9-3 is a private method that will return `__Node` objects for both the target node and its parent. This will be used for `search()` and `insert()` operations, which differ in whether full nodes should be split during the search process. The `prepare` parameter tells the method whether or not to split those nodes during the search.

Listing 9-3 *Constructor, Basic, and `__find()` Methods of the `Tree234` Class*

```
class Tree234(object):          # A 2-3-4 multiway tree class
…
   def __init__(                # The 2-3-4 tree organizes items in
         self):                 # nodes by their keys.
      self.__root = None        # Tree starts empty.

   def isEmpty(self):           # Check for empty tree
      return self.__root is None

   def root(self):              # Get the data and keys of the root node
      if self.isEmpty():        # If the tree is empty, raise exception
         raise Exception("No root node in empty tree")
      nKeys = self.__root.nKeys # Get active key count
      return (                  # Otherwise return root data and key
         self.__root.data[:nKeys], # arrays shortened to current
         self.__root.keys[:nKeys]) # active keys

   def __find(self,             # Find a node with a key that matches
              goal,             # the goal and its parent node.
              current, parent,  # Start at current and track its parent
              prepare=True):    # Prepare nodes for insertion, if asked
      if current is None:       # If there is no tree left to explore,
         return (current, parent) # then return without finding node
      i = 0                     # Index to keys of current node
      while (i < current.nKeys and # Loop through keys in current
```

```
            current.keys[i] < goal): # node to find goal
        i += 1
    if (i < current.nKeys and   # If key i is valid and matches goal
        goal == current.keys[i]):
        return (current, parent) # return current node & parent
    if (prepare and         # If asked to prepare for insertion and
        current.nKeys == Tree234.maxKeys): # node is full,
        current, parent = self.__splitNode( # then split node, update
            current, parent, goal) # current and parent, and adjust i
        i = 0 if goal < current.keys[0] else 1 # for new current
    return ((prepare and current, parent) # Return current if
            if current.isLeaf() else # it's a leaf being prepared
            self.__find(    # Otherwise continue search recursively
                goal,       # to find goal
                current.children[i], # in the ith child of current
                current, prepare))   # and current as parent
```

By using the same routine to perform both searching and inserting, `__find()` must perform differently on duplicate keys depending on the operation. Searching for an existing key and inserting a duplicate key both call `__find()` with a `goal` that matches some key in the tree. For a search, `__find()` should locate the node with the key and return its associated data value. It doesn't need to split any full nodes because the tree isn't being altered.

For inserts, `__find()` should split the full nodes as the search descends the tree. As noted earlier, splitting full nodes while descending from the root maintains the tree's balance. If `__find()` starts an insert operation but ends up finding a duplicate of the `goal` key, it will update the data for that key. This makes the 2-3-4 tree behave as an *associative key store* for duplicate keys.

The `__find()` routine recursively descends the 2-3-4 nodes. It starts at the `current` node and tracks its `parent`. The caller normally passes the root node with `parent` pointing at the `Tree234` object itself. If `current` is `None`, `__find()` immediately returns it and the parent pointer, handling the base recursion case. When there is a `current` node, the first step is to determine where among the existing keys the `goal` key lies.

After setting index `i` to 0, the lowest key index, the `while` loop goes through all the valid keys, stopping when the `i`th key is equal to or greater than the `goal`. Because the sorted array contains at most three keys, using a binary search for the goal key wouldn't save more than one comparison. After the loop, if the `goal` was found as a valid key, it can return the current node and its parent. Note that even if this node is full and `__find()` is preparing for an insert,

there's no need to split it. Updating the data for a duplicate key doesn't require a split.

The next `if` statement handles splitting up the full nodes. When the `prepare` flag is true and the number of keys is the maximum allowed, it calls the `__splitNode()` method to redistribute those key-data pairs into separate nodes. Performing that split can change where the `current` and `parent` pointers should be, so `__find()` updates those variables from the multiple results of the call. Remember that splitting the root requires creating a new parent of the node being split, and that parent becomes the new root. Because the key-data pairs get moved, it also needs to update the `i` index in the `current` node. That `current` node will have only one key left in it (when split by the `__splitNode()` method, as you see shortly), so it only needs to check whether the goal is before or after the first key. When the goal is before, the `i` index is set to 0; otherwise, it is set to 1.

At this point, `__find()` hasn't found the goal, and it has performed any requested split. The search should continue in one of the child nodes if they exist. There are several cases here. If the `current` node is a leaf, then there are no children, and this is the node where the insertion should occur or the search should terminate. The complex `return` statement first checks if `current.isLeaf()` returns true. If so, it returns the `current` and `parent` nodes with one additional check. By returning `prepare and current` as the first result, `__find()` ends up returning `False` when the call is part of a search operation and returning `current` as part of an insert. Checking the `prepare` flag and conditionally returning `current` handles both kinds of operations on leaf nodes.

If the `current` node is not a leaf, then both searches and inserts must continue in a child node. The exact child depends on where the `while` loop finished. If the `goal` is less than the lowest key, `i` will be 0 and the lowest indexed child will be the next to explore. Every other key that was checked has incremented `i` and moved on to the next link in the `children` array. If all three keys were checked, `i` ends on the last child. The `__find()` method recursively calls itself with the i^{th} child. The `parent` pointer for that child is the `current` node as it descends the tree. If it had to split the current node, then it adjusted the `i` index to point at the appropriate child before making the recursive call.

Splitting Full Nodes

The method for splitting nodes containing three items, `__splitNode()`, is shown in Listing 9-4. The parameters are a reference to the node to be split, `toSplit`, its `parent` node, and the `goal` key that is being inserted in the tree. It handles splits for all three situations: the root node, an internal node, and a leaf node. All situations require making at least one new node that contains key (and data) 2. That `newNode` is a leaf node if the node `toSplit` is also a leaf node. The first `if` statement checks the node's position in the tree and passes the highest two child links to the `__Node()` constructor if they exist. These are the child links just below and above key 2.

Listing 9-4 *The `__splitNode()` method of `Tree234`*

```
class Tree234(object):          # A 2-3-4 multiway tree class
…
   def __splitNode(             # Split a full node during top-down
         self,                  # find operation.
         toSplit,               # Node to split (current)
         parent,                # Parent of node to split (or tree)
         goal):                 # Goal key to find
      if toSplit.isLeaf():      # Make new node for Key 2, either as a
         newNode = self.__Node( # leaf node
            toSplit.keys[2],    # with key 2 and value 2 as its
            toSplit.data[2])    # sole key-value pair
      else:
         newNode = self.__Node( # or as an internal node
            toSplit.keys[2],    # with key 2 and value 2 as its
            toSplit.data[2],    # sole key-value pair and the highest
            *toSplit.children[2:toSplit.nChild]) # 2 child links
      toSplit.nKeys = 1         # Only key 0 and data 0 are kept in
      toSplit.nChild = max(     # node to split and child count is
         0, toSplit.nChild - 2) # either 0 or 2
      if parent is self:        # If parent is empty (top of 2-3-4 tree)
         self.__root = self.__Node( # make a new root node
            toSplit.keys[1],    # with key 1 and value 1
            toSplit.data[1],    # and the node to split plus new node
            toSplit, newNode)   # as child nodes
         parent = self.__root   # New root becomes parent
      else:                     # For existing parent node,
         parent.insertKeyValue( # insert key 1 in parent with
            toSplit.keys[1],    # new node as its higher subtree
            toSplit.data[1], newNode)
      return (toSplit           # Find resumes at node to split if goal
              if goal < toSplit.keys[1] # is less than key 1
```

```
                else newNode,    # else new node
                parent)          # Parent is either new root or same
```

After `__splitNode()` creates the `newNode`, the next statements update the number of keys and children of the node `toSplit`. That node will contain only key (and data) 0 after the split. The method leaves the references to the other keys, data, and child links in the arrays because it needs to reference key 1 shortly. Ideally, the array cells should be set to `None` to erase the extra references. Keeping only key 0 means the `toSplit` node now has either two or no child links, depending on whether it is a leaf node.

After `__splitNode()` alters the number of keys and child links, the next `if` statement checks whether the node `toSplit` is the root node by testing if its `parent` is the `Tree234` object itself. Splitting the root node requires making a second new node to serve as the root of the tree. The new root holds key (and data) 1 of the node `toSplit`. The new node becomes the parent of the node `toSplit`, which contains key 0, and the `newNode` containing key 2.

With the creation of the new root, the `parent` reference passed from the `__find()` method is set to that root. That `parent` pointer will be returned shortly, so the caller can recognize the new parent node.

If the node `toSplit` is not the root node, then it must be an internal or leaf node and `__splitNode()` should insert its key (and data) 1 into the existing `parent` node by calling `insertKeyValue()`. Remember that the parent cannot be full because it would have been split on the previous call to the `__find()` method, which was the call on the `parent` node. You also know that the subtree holding keys just above key 1 is rooted at the `newNode` created earlier, holding only key 2.

At this point, after the second `if` statement, all the data items that were stored at the node `toSplit` have been placed in their new nodes, and those nodes have been linked to one another. It's time to return to the `__find()` method and update its `current` and `parent` pointers to where the search should continue. The `current` pointer should point to the subtree that contains the `goal`. That means the subtree to return is based on the relationship of the `goal` to the keys that were split. If the `goal` key is less than key 1 of the node `toSplit`, then the search resumes with that same node because it contains key 0 and any subtrees below it. Similarly, if the `goal` key is greater than key 1, the search should continue with the `newNode` that was created containing key 2 and its child links, if any. The `goal` key cannot match key 1 because that would mean it's a

duplicate handled by the `__find()` method. The return value of `parent` is simple because it is always the variable, `parent`, although that could have been modified if the node `toSplit` was the root node.

Listing 9-5 *The* `search()` *and* `insert()` *Methods of* `Tree234`

```
class Tree234(object):          # A 2-3-4 multiway tree class
…
   def search(self, goal):      # Public method to get data associated
      node, p = self.__find(    # with a goal key. First, find node
         goal, self.__root,     # starting at root with self as parent
         self, prepare=False)   # without splitting any nodes
      if node:                  # If node was found, find key in node
         return node.data[      # data. It's the first data (index 0) if
            0 if node.nKeys < 2 or # there's only 1 key or
            goal < node.keys[1] else  # if the goal < key 1, else it's
            1 if goal == node.keys[1] # the 2nd data if goal == key 1
            else 2]                   # Otherwise it's the 3rd data

   def insert(self,             # Insert a new key-value pair in a
              key,              # 2-3-4 tree by finding the node where
              value):           # it belongs, possibly splitting nodes
      node, p = self.__find(    # First, find insertion node for key
         key, self.__root,      # starting at root with self as parent
         self, prepare=True)    # and splitting full nodes
      if node is None:          # If no node was found for insertion
         if p is self:          # Check if this the root
            self.__root = self.__Node( # Make a root node with just
               key, value)      # 1 key value pair
            return True         # and return True for node creation
         raise Exception(       # If not root, then something is wrong
            '__find did not find 2-3-4 node for insertion')
      return node.insertKeyValue( # Otherwise, insert key in node
         key, value)            # with no subtree, returning insert flag
```

You now have all the modules needed to define the public `search()` and `insert()` methods as shown in Listing 9-5. Both start by calling `__find()` on the root to get a pointer to the node that either holds or should hold the `goal` key. The `search()` method passes `False` for the `prepare` flag to avoid splitting nodes and allowing `__find()` to return `None` or `False` if the `goal` isn't found. If a `node` is returned, then one of its keys must match the `goal`. The `return` statement of `search()` selects one of the three `node.data` items by checking

the quantity of keys in the `node` and the relationship of its key 1 to the `goal`. If no node was returned from `__find()`, then `search()` also returns `None`.

The `insert()` method is almost as simple. If no `node` was returned from `__find()`, then the tree should have no leaf (or other) nodes. That's different from the find methods you've seen in other data structures, but remember, the `__find()` method will split nodes to make room for the key to insert. The second `if` statement checks the parent, `p`. If it is the `Tree234` object, then this is the first key being added to an empty tree. The root node of the tree is set to be a new `__Node` holding the `key` and `value` to insert with no children. It returns `True` to indicate that a new item was inserted. If the parent was something other than the `Tree234` object and `node` is `None`, then a bug has occurred, and it raises an exception. This could only happen if the `__find()` method started descending through some 2-3-4 nodes and ended up returning `None` for the insertion node.

When `__find()` returns a nonempty 2-3-4 node, the `insert()` method inserts the key-value pair in that node. The `insertKeyValue()` method returns `True` if a new key is inserted and `False` if an existing key is updated. The `insert()` method returns that flag to its caller.

As you can see, insertion in 2-3-4 trees is quite a bit more complicated than in binary search trees or ordered linked lists. If the code confuses you in spots, use the Visualization tool and step slowly through the confusing spots. Seeing the code alongside the data structure as it is updated can help clarify the algorithm and purpose of each line of code.

Traversal

Traversing a 2-3-4 tree adds some new wrinkles to the possible tree traversal orders. The in-order traversal ought to visit each of the data items in order of ascending keys, similar to binary search trees. That's easy to do by first traversing child 0, and then alternating between the keys in the node and their corresponding "right" child. In terms of the child and key names for a node, that would be child 0, key 0, child 1, key 1, child 2, key 2, and child 3.

It's less clear what a pre-order or post-order traversal should do. One option for pre-order traversal would be to visit all the key-value pairs stored in a node before visiting any of its children. Another would be to visit the smallest key before visiting the first two children to mimic the behavior of a binary tree, and

then visit key 1 before child 2 and key 2 before child 3. That approach somewhat preserves the in-order behavior for keys 1 and 2. A third pre-ordering would visit the first key and the first child, then the second key and the second child, followed by the third key and the third and fourth child. That last ordering is shown in Figure 9-10 along with the symmetrical ordering for post-order traversal.

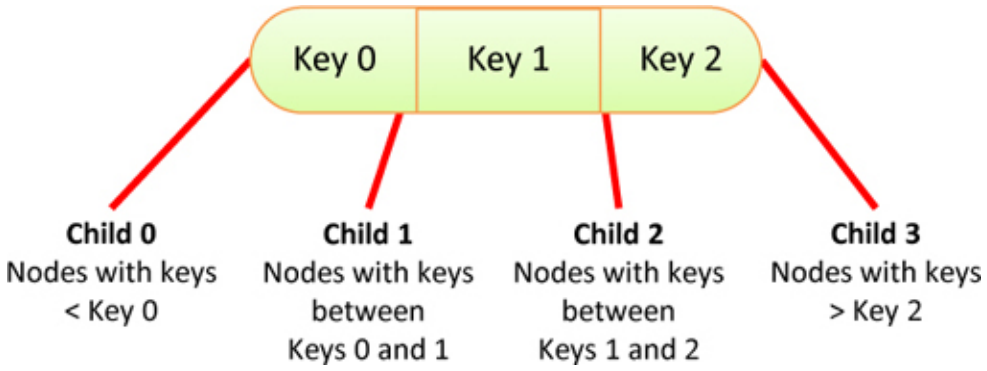

Figure 9-10 *Traversing 2-3-4 trees*

The exact ordering to implement depends on the planned uses of the data structure. An implementation of the traversal orders of Figure 9-10 in the `traverse()` generator is shown in Listing 9-6. Like with the binary search trees, implementing this using a nonrecursive generator is the most efficient way to yield keys paired with their data.

At the start, the `traverse()` method raises an exception if the requested `traverseType` is not one of the expected values: pre, in, or post. Next it creates a stack, using the linked list stack defined in `LinkStack` from Chapter 5, "Linked Lists." The stack is initialized to hold the 2-3-4 tree's root node. These are the same steps as in the `traverse()` method used in binary search trees.

Listing 9-6 *The `traverse()` Generator for `Tree234`*

```
from LinkStack import *

class Tree234(object):          # A 2-3-4 multiway tree class
…

   def traverse(self,           # Traverse the tree in pre, in, or post
                traverseType="in"): # order based on type
      if traverseType not in [ # Verify traversal type is an
```

```
            'pre', 'in', 'post']: # accepted value
        raise ValueError(
            "Unknown traversal type: " + str(traverseType))

    stack = Stack()          # Create a stack
    stack.push(self.__root) # Put root node in stack

    while not stack.isEmpty(): # While there is work in the stack
        item = stack.pop()    # Get next item
        if isinstance(item, self.__Node): # If it's a tree node
            last = max(       # Find last child or last key index
                item.nChild,  # going 1 past last key for post order
                item.nKeys + (1 if traverseType == 'post' else 0))
            for c in range(last - 1, -1, -1): # Loop in reverse
                if (traverseType == 'post' and # For post-order, push
                    0 < c and c - 1 < item.nKeys): # last data item
                    stack.push((item.keys[c - 1], item.data[c - 1]))
                if (traverseType == 'in' and # For in-order, push
                    c < item.nKeys): # valid data items to yield
                    stack.push((item.keys[c], item.data[c]))
                if c < item.nChild: # For valid child links,
                    stack.push(item.children[c]) # traverse child
                if (traverseType == 'pre' and # For pre-order, push
                    c < item.nKeys): # valid data items to yield
                    stack.push((item.keys[c], item.data[c]))
        elif item:            # Every other non-None item is a
            yield item        # (key, data) pair to be yielded
```

The `while` loop processes items from the stack until it is empty. It pops the top `item` off the stack and looks at its type to determine what to do next. Like the `traverse()` method for binary search trees, there can be three types of items—a 2-3-4 node, a (key, data) tuple, or a `None` value—which occur for empty 2-3-4 trees or when leaf nodes push their child links on the stack.

When the item to process is a 2-3-4 node, the various keys and child links must be pushed onto the stack in an order that depends on the traverse type. The number of keys and child links is different for each node, and leaf nodes differ from internal nodes in the number relationship. The method determines the `last` index of either a child node or key that it will need to process using the maximum of the number of children and the number of keys. For an internal node, the loop must cover all the child nodes. For a leaf node, the loop must cover all the stored items. In the case of post-order traversal, the `last` index could be as high as the number of keys plus one, which is the same as the

number of child links for internal nodes but not for leaf nodes. Adding one helps simplify the loop that comes next.

The `for` loop increments a variable, `c`, over the range 0 through `last - 1` in reverse order. This loop differs from what you've seen in other data structures, but it's needed to handle all the different types of nodes and traversal orders. The reverse ordering makes the items pushed on the stack come out in the desired order when they are popped by the outer `while` loop.

Inside the loop, four `if` statements control the order of visiting the keys and child links. The first one handles post-order traversal. This is the most complex of the three orders because you want to process the last key after the last child. In this case you want to push the `c - 1` key on the stack so it is visited after child `c`. For instance, in a full 2-3-4 node, the last key, key 2, must be pushed on the stack before pushing on the last child, child 3. The stack items are processed in the reverse order, yielding key 2 after processing child 3. The`if` statement verifies that `c - 1` refers to a valid key before pushing a tuple with that key and its corresponding data onto the stack.

The second `if` statement handles the case of in-order traversal. In this case, the `c` key should be visited immediately after child `c`. If `c` refers to a valid key in the node, the `c` key and `c` data are pushed as tuple on the stack.

The third `if` statement handles the processing of child `c`. Depending on the traversal type, you could have already pushed on the (key, data) pair for either the `c - 1` key or the `c` key. When the method pushes child `c` on the stack now, that child is processed immediately before that key. This `if` statement checks that `c` refers to a valid child before pushing it on the stack (although it would be acceptable to push a `None` value). The type of this item is always a 2-3-4 node, which can be distinguished from the (key, data) tuples.

The fourth and final `if` statement handles the case of pre-order traversal. In this case, child `c` was just pushed on the stack, so the method pushes key `c` and its data to be processed before that child.

Traversal is usually the simplest operation of a data structure, but the various types of nodes and numbers of items within them complicate the process for 2-3-4 trees. The visualization tool shows the execution of the three orderings along with the stack contents described here to help clarify the details.

Deletion

We showed how the insertion method can maintain a balance as items are inserted in the 2-3-4 tree. It would be nice if deleting a node did the same. Unfortunately, deleting while maintaining balance is quite a bit more complex than insertion. We review how you can accomplish this task but skip the detailed implementation for the sake of brevity.

Deleting at Leaf Nodes

Let's first consider the easy case. If you delete an item from a leaf node with more than one item in it, nothing needs to be done other than shifting items in the arrays of keys and data. There is still at least one item in the node, and its key remains in proper relation with the keys in the parent and sibling nodes. More importantly the number of nodes and their levels are identical, so balance is maintained.

There's one more easy case for a leaf node. When you delete the only item in the root node and the root is also a leaf node, then it must be the last item in the tree. Deleting it means the tree is now empty. Deleting leaf nodes elsewhere in the tree might cause imbalances and we shall soon see.

Deleting at Internal Nodes

Now let's consider a deletion from an internal node. Deleting an item from the node would mean reducing the number of child nodes. That's possible in some circumstances but not always easy. You might be able to combine the two subtrees on either "side" of the key being deleted, but that could be messy. Is there another way? Remember deletion in binary search trees? We saw some simple cases and the more difficult case when the node had two children. Do you remember the "trick" for that case?

The idea is that you could replace the item to be deleted by *promoting its successor*. The successor item has the next higher key in the tree. There must be one because you're deleting from an internal node that has at least two child nodes. Finding the successor item is almost as simple as it was in the binary search tree. Start in the child node just to the "right" of the key being deleted. If you're deleting the i^{th} key of a node, you start looking for the successor in child $i + 1$. This is the subtree containing keys larger than the i^{th} key. Then you follow any child 0 links until you reach a leaf node. The lowest key in the leaf node identifies the successor.

Figure 9-11 shows an example of deleting the item with key 15 from a 2-3-4 tree. After locating the node holding keys 15 and 30, you find that 15 is key 0 and it's an internal node. That means the successor must be the smallest key in the subtree that is child 1 of that node. The child, node 19-22-26, is a leaf node so you don't have to descend any more levels (through the child 0 links), and the successor is the smallest key in that leaf node, 19. If you put item 19 in the node that was holding items 15 and 30, then you can delete item 19 from the leaf node. That's the simple case you already know how to handle. The bottom of Figure 9-11 shows the tree after deleting item 15. Item 19 was promoted to fill the hole created by deleting item 15, and items 22 and 26 were shifted left in the leaf node.

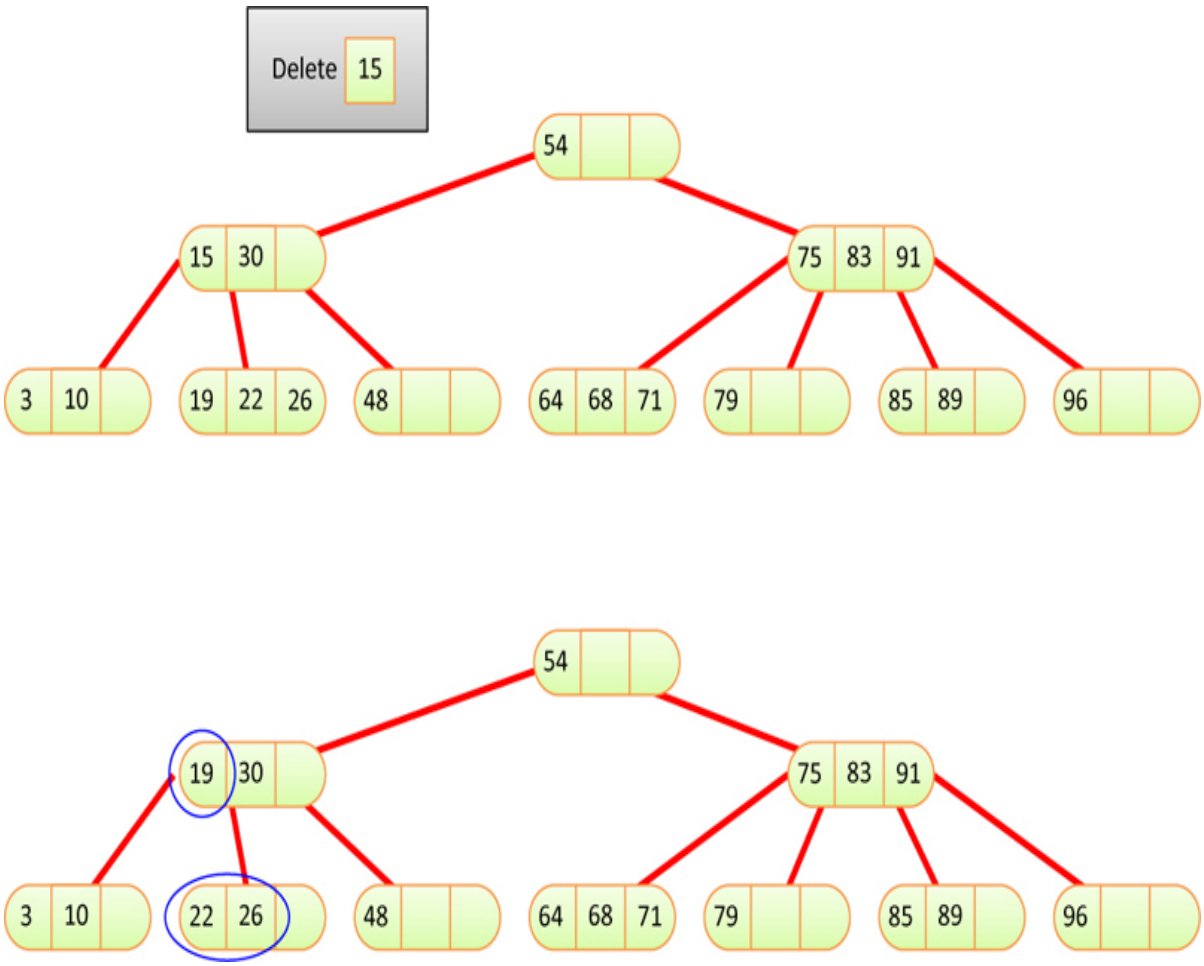

Figure 9-11 *Deleting from an internal node by promoting the successor*

To delete an item from an internal node whose successor lies in a leaf node with other items, you can now replace the item followed by the simple deletion of the item in the leaf. The tree maintains the same number of nodes and levels,

so balance is maintained. Does this strategy work for all items in all internal nodes?

Consider deleting item 30 in the initial tree of Figure 9-11. This time you need to hunt for the successor in child 2 of node 15-30. The successor is 48, but there's a problem. Item 48 is the only item in the leaf node. If you promote it to replace 30, and then delete it from the child node, you will have an empty 2-3-4 node, and that's not allowed. Is there another way?

In this case, yes, there is. Just as there is a successor item, there is also a *predecessor* item for every item in an internal 2-3-4 node. Finding the predecessor is the mirror of finding the successor. The search starts in the i^{th} child and then follows the maximum child links in that subtree, if any, until it reaches a leaf node. In the leaf node, the maximum key identifies the predecessor item. If the predecessor is not the only item in the leaf node, you can promote it to replace the target item—key i—and delete the predecessor. In the example of deleting item 30, this ends up being a promotion of item 26 followed by the easy case of deleting 26 from leaf node 19-22-26.

Deleting from 2-Nodes

The technique of promoting a successor or predecessor is pretty powerful. All six of the items in internal nodes of the initial tree of Figure 9-11 have a successor or a predecessor that lies in a leaf node with more than one item. Unfortunately, you cannot count on the predecessor or successor to always lie in a node with other items. You need some other techniques to deal with nodes with only one item, also called 2-nodes because they have two child links, and with leaf nodes holding a single item.

There are two techniques to solve this problem. Consider deleting item 54 from the tree shown in Figure 9-12. By following the links, you can see that both the predecessor and successor contain only one item (and can be considered 2-nodes). Now you can't simply choose one of them to replace item 54 because deleting the single item in the leaf would break the rules of 2-3-4 trees.

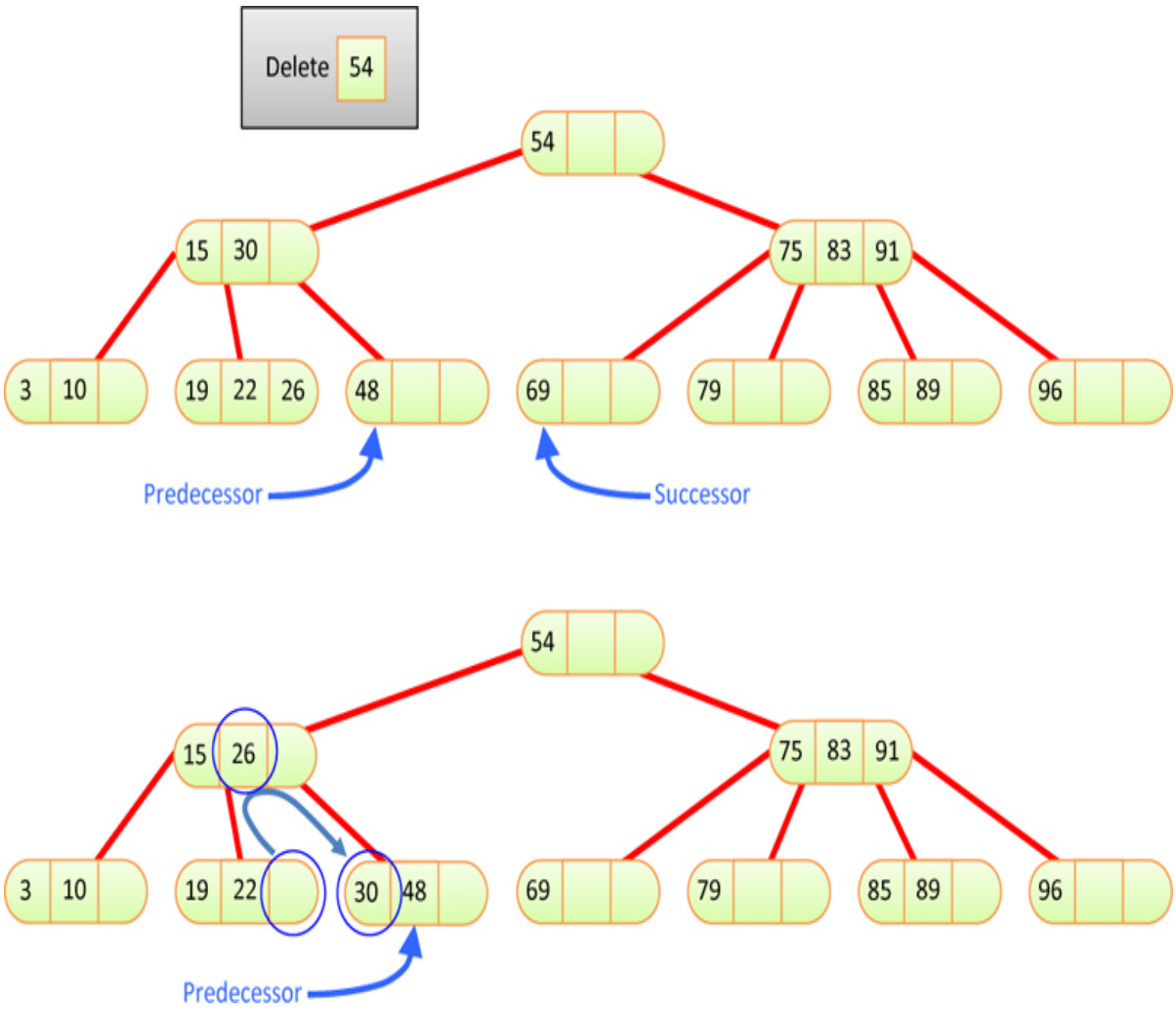

Figure 9-12 *Deleting an item by rotating items into the predecessor node*

To change the situation for the predecessor, we can look at its relationship to its sibling nodes. The predecessor in this case has one sibling holding three items (which can be considered a 4-node). Can we "borrow" one of them to make the predecessor have two? Well, not exactly, but we can shift the sibling's item up and move an item down from the predecessor's parent. This is shown in the bottom tree of Figure 9-12. You move item 26 from the sibling to the parent and item 30 from the parent to predecessor. The circles show where items changed.

Moving these two items is called a **rotation**. The rotation reconfigures the tree without breaking any of the rules of 2-3-4 trees. You still have the items stored in sorted order, and the tree is still balanced. By rotating item 26 up and item 30 down, you've created a tree where you can do the simple deletion: remove

predecessor item 48 from the leaf node and replace item 54 with item 48. That final step is not shown in Figure 9-12.

Rotations can be performed whenever the predecessor or successor has a sibling that is either a 3-node or a 4-node (when it has two or three items stored in it). Depending on which side the sibling lies, the rotation is either to the left or the right (shifting higher keys to the lower sibling or lower keys to the higher sibling). The parent can be any kind of node, 2-node, 3-node, or 4-node, because you are simply changing one item in it.

What if neither the predecessor nor the successor has a 3-node or 4-node sibling? That's what happens with the successor of item 54 in Figure 9-12; the successor item has key 69 and its only sibling is a 2-node containing item 79. Figure 9-13 shows the second technique that applies to this situation.

Figure 9-13 *Deleting an item by fusing items into the successor node*

When the 2-node that you're trying to fix only has siblings that are also 2-nodes, you can **fuse** the single item of the successor with that of a sibling and an item from their common parent, as long as the parent has two or three items (is a 3-node or 4-node). In the figure, the successor to item 54 is item 69, a 2-node. The only sibling to the successor is also a 2-node containing the single item 79. Because their common parent, node 75-83-91, is a 4-node, you can steal the item that separates the two siblings, item 75, and put it together with the single items to form a new 4-node, node 69-75-79, as shown in the lower tree. This is called a **fusion** operation.

As Figure 9-13 shows, fusing the two single item siblings with an item from their parent eliminates a node from the tree. That sounds like it might affect the

balance, but if you look closely, it doesn't. The items remain in sorted order, and the number of levels hasn't changed on any path. The path to the deleted node went away, so it doesn't matter anymore. One item was removed from the parent of the successor, but that's fine because the corresponding child link was removed.

Using both rotation and fusion, you can make either a successor or a predecessor node into a multi-item node and then perform the deletion after replacing the item to delete higher in the tree. Although we've shown rotation on the predecessor and fusion on the successor, they really can be applied on either side. What matters is the number of items stored at the nodes and their parent.

Extending Fusion

Is that all the cases? Will rotation and fusion solve everything? The answer is almost. We need to address a few more special cases, but they turn out to be variations on what you've already seen. An exception that doesn't fit any of the rules you've seen is a full binary tree. If every node in a 2-3-4 tree has exactly one item, then every node is a 2-node, like the top example shown in Figure 9-14.

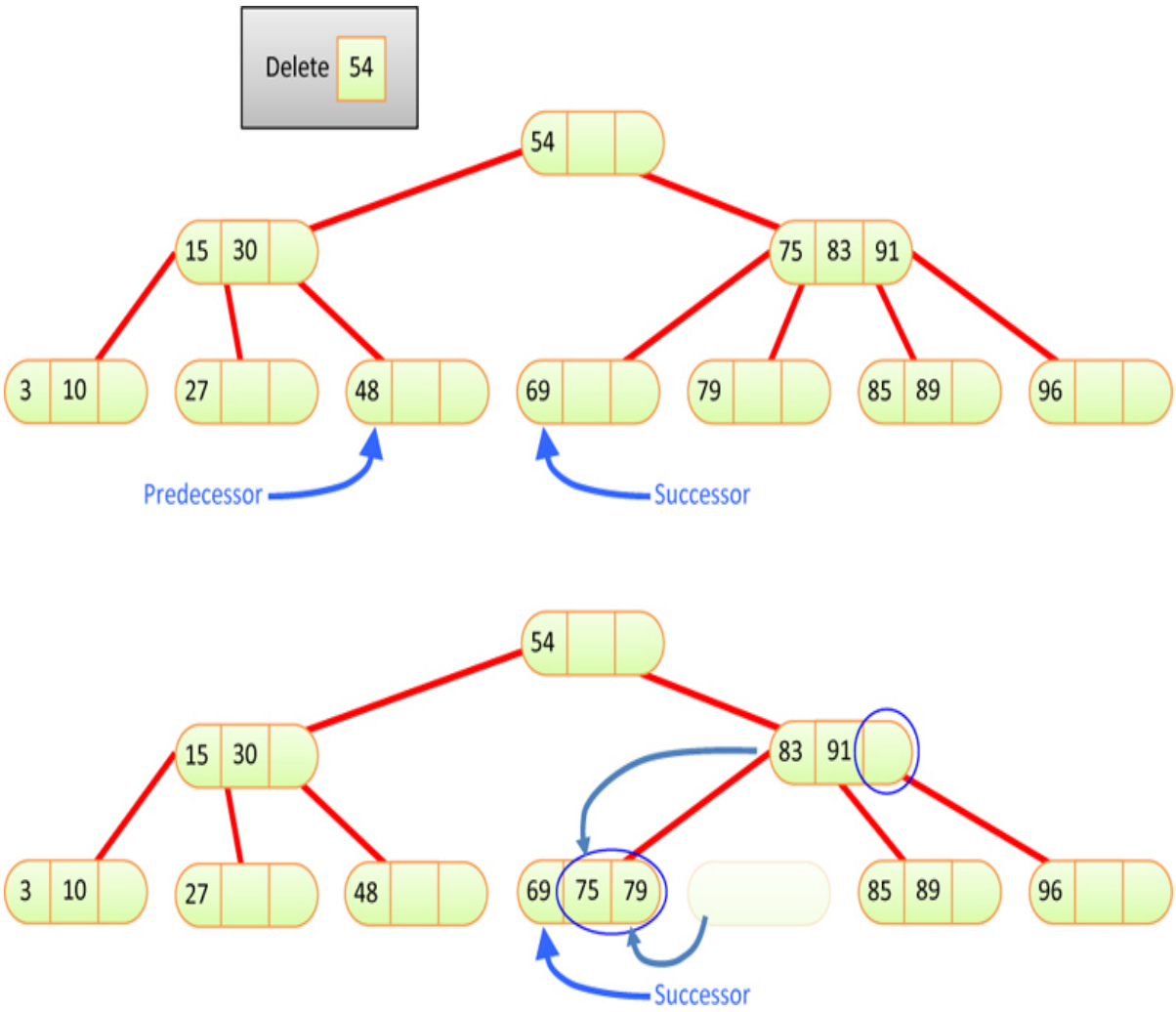

Figure 9-14 *Deleting from a 2-3-4 tree with only 2-nodes*

When every node is a 2-node, you can't use rotation or fusion as described earlier because both need a sibling or parent node to be either a 3-node or a 4-node. You obviously need some other technique to deal with this situation. There's another limitation of the operations described so far: the tree never shrinks. Each of the cases you've seen so far keeps the height of the tree (number of levels in a path to a leaf) the same. Replacing the item to delete with its successor and then deleting the successor from the leaf node leaves the same number of levels. Rotation and fusion also leave the same number of levels. Even deleting the last item from the root node that is also a leaf node technically changes the number of levels from zero to zero. How can the tree ever shrink from having two levels to one level with these techniques?

You can solve both limitations with a simple change to the fusion operation. If you allow fusion to occur when the parent is a *2-node that is also the root node*

and replace the root node by the fused node, then the tree shrinks by one level. Fusion already eliminates a node; now you are extending it to eliminating a whole level. By limiting this fusion to only happen on the root node, it guarantees that all paths in the tree shrink by one level and preserves balance.

In the example in Figure 9-14, the top tree is transformed into the middle tree by fusing the top three nodes and their single items into one 4-node. The item at the root with key 62 becomes the middle item of the new 4-node. The items at the root's two children become item 0 and item 2 of the new 4-node. Child 0 and 1 of node 30 become child 0 and 1 of the 4-node. Child 0 and 1 of node 83 become child 2 and 3 of the 4-node. In other words, extended fusion collapses the top three 2-nodes into a single 4-node, removing one level of the tree.

From the middle tree in Figure 9-14, you can now delete item 62 using the operations you've already seen. Let's now focus on item 62's predecessor or successor. Both are 2-nodes, and both have only 2-nodes as siblings. Their parent, the new root, is a 4-node, so you can apply the fusion operation. The lowest tree in the figure shows the application of fusion to nodes 46 and 71 to make a new 4-node containing 46-62-71. This is one more special aspect of the fusion operation: it may move the item to be deleted into a lower position, possibly a leaf node to be deleted easily. In this case, item 62 started at level 0. The first fusion left it as item 1 of the new root, which is still level 0. The second fusion moved it to level 1, from which it can be deleted without reconfiguring nodes.

Applying Rotation and Fusion on Descent

It looks as though you now have all the rules to transform the tree and use simple deletion from leaf nodes. There is, however, one more thing to add in how to apply these operations. Consider what would happen if you asked to delete item 83 from the top tree in Figure 9-14. If you simply skip past the root, node 62, and then try to delete item 83 from the right subtree, you'll get stuck again because both the successor and predecessor are 2-nodes with a 2-node for a parent. Because the parent of the predecessor and successor isn't the root, you can't use the extended fusion operation (if you did, it would shorten the paths for only the leaves of this subtree without shortening the paths of all leaf nodes equally). Now what?

The problem here is that you didn't recognize at the root that you would need to apply a fusion operation below. If you had, you would end up with the

middle tree in Figure 9-14, which makes it possible to delete 83 after another fusion operation. How can you know whether that fusion will be needed or not? The answer resembles what we did for insertion into the tree. Remember that as we followed the path to the insertion point, we split nodes that were full. That approach ensured that when we arrived at the insertion point, the parent node was not full and could accept another item. For insertion, we assumed we would need the full nodes to be split. What happens if you assume that you need 2-nodes to be collapsed as you descend the tree to hunt for the node to be deleted and its successor or predecessor?

When deletion is implemented for 2-3-4 trees, it requires an algorithm like the `__find()` method with the `prepare` flag that looks at every node encountered along the path and applies rotation or fusion to change any 2-node into a 3-node or a 4-node. In other words, you simply assume that nearly empty 2-nodes need to be collapsed. This algorithm also must track both the item to find and delete and the successor item as nodes are rearranged. All these extra details make it quite a bit more complicated than the insertion routine. There are many cases to examine with sibling nodes and the parent node, and a lot of rearranging of items and child links. In the rotation example we showed, the items being rotated were all at the leaf level and one above. When you apply rotation on internal nodes, the subtrees associated with the two deeper items being rearranged must follow those items. We look at that topic in more detail when we discuss rotation in red-black trees in Chapter 10, "AVL and Red-Black Trees."

The key points to remember about deletion in 2-3-4 trees are

- The tree can remain balanced as items are deleted.
- Deletion uses a modified version of the `__find()` method to locate the item to delete by descending through the tree comparing keys.
- After you find the item to delete, a similar descent of the tree is used to find the predecessor and successor of the item.
- Both descents apply rotation and fusion operations to 2-nodes encountered along the way to ensure that when the item to delete or the predecessor/successor is located, it's in a node with more than one item (or it's the root node with a single item).

Efficiency of 2-3-4 Trees

Fully analyzing the efficiency of 2-3-4 trees is hard, but they bear a lot of similarities to the binary trees. We can certainly determine the Big O complexity of operations and memory usage, and that's what's most important.

Speed

With the binary trees you saw in Chapter 8, one node on each level must be visited during a search, whether to find an existing node, insert a new one, or delete one. The number of levels in a balanced binary tree is about $\log_2(N)$, so search times are proportional to this.

One node must be visited at each level in a 2-3-4 tree as well, but the 2-3-4 tree is shorter (has fewer levels) than a binary tree with the same number of data items. To see this, compare the transformed trees in Figure 9-5 and Figure 9-14. In both cases, a 2-3-4 tree has been split or collapsed into a (partial) binary tree.

More specifically, in 2-3-4 trees there are up to four children per node. A full, single-node 2-3-4 tree has 3 items and height 0. A full 2-3-4 tree of height 1 has 5 nodes and 15 items. If every node were full, the height of the tree would be proportional to $\log_4(N)$, where N is the number of items (not nodes). Logarithms to the base 2 and to the base 4 differ by a constant factor, $2 \times \log_4(x) = \log_2(x)$ or $\log_4(x) = ½ \log_2(x)$. Thus, the height of a 2-3-4 tree would be about half that of a binary tree, provided that all the nodes were full. Because they aren't all full, the height of the 2-3-4 tree is somewhere between $\log_2(N)$ and $\log_2(N)/2$. The reduced height of the 2-3-4 tree somewhat decreases the path to search compared with binary trees.

On the other hand, there are more items to examine in each node along the path, which increases the search time. Examining the data items in the node using a linear search multiplies the search time by an amount proportional to M, the average number of items per node. Even if a binary search is used on the sorted keys in a node, you end up with a search time proportional to $M \times \log_4(N)$ because a binary search only saves one comparison compared to a linear search and only when there are three keys to compare.

Some nodes contain one item, some two, and some three. If you estimate that the average is two, search times will be proportional to $2 \times \log_4(N)$. If you convert that to logarithms of base 2, you get $2 \times \log_4(N) = 2 \times \frac{1}{2} \log_2(N) = \log_2(N)$. Thus, for 2-3-4 trees, the increased number of items per node tends to cancel out the decreased height of the tree. The search times for a 2-3-4 tree and for a balanced binary tree are approximately equal, and are both O(log N).

The story is similar for insertions and deletions. They each descend through the $\log_{4(}N)$ levels. Both insertions and deletions need to reach the leaf level to do their work. As they descend, the insertion process splits full 4-nodes and the deletion process collapses 2-nodes into 3-nodes or 4-nodes. These operations consume time, but the amount of time does not depend on the number of nodes in the tree; it's roughly constant for each node along the path. It does depend on the number of items per node, M. The node modification operations can be thought of as another constant factor, C, which ends up making the time to insert or delete $C \times M \times \log_4(N)$. That's still O(log N).

Traversal, of course, must visit every node, so its overall time is O(N).

Storage Requirements

Each node in a 2-3-4 tree contains storage for three keys and references to data items and four references to its children. This space may be in the form of arrays, as shown in `Tree234.py`, or of individual variables. Most 2-3-4 trees, however, do not use all this storage. A 2-node with only one data item will waste 2/3 of the space for data and 1/2 the space for children. A 3-node with two data items will waste 1/3 of the space for data and 1/4 of the space for children. If the spaces taken for a key, data, and child reference are identical and there are two data items per node as the average utilization, then 4 of the 6 cells for keys and data are used and 3 of 4 cells for child links are used, on average. That's 7 of 10 in use and 3 of 10 (30%) wasted.

You might consider using linked lists instead of arrays to hold the child and data references, but the time overhead of the linked list compared with an array, for only three or four items, would probably not make this a worthwhile approach.

The number of nodes is not the same as the number of items in a 2-3-4 tree. The number of nodes could be as low as 1/3 the number items if all the nodes

are full (4-nodes). At the other extreme, there can be only one item per node (2-nodes), so the number of nodes ranges from N/3 to N, with the average expected to be N/2 items. In Big O notation, you treat all of those as O(N). That also means that the average amount of unused memory is O(N).

As traversals are performed, the algorithm needs to store information for each level of the descent of the tree. This storage can either take the form of recursive calls or an explicit stack of nodes that need to be visited. That stack will grow to the number of levels of the tree, so the memory needed is O(log N). Note that search, insertion, and deletion don't need recursion (although the implementation of `__find()` in `Tree234` did use it); they can be written to follow the chain of pointers without storing the full path back to the root. That means they need only O(1) space.

2-3 Trees

We discuss 2-3 trees here because they are historically important. Also, some of the techniques used with 2-3 trees are applicable to B-trees, which we examine in the next section. Finally, it's interesting to see how a small change in the number of children per node can cause a large change in the tree's algorithms.

There are many similarities between 2-3 trees and 2-3-4 trees except that, as you might have guessed from the name, they hold one fewer data item and have one fewer child. They were the first multiway tree, invented by J. E. Hopcroft in 1970. B-trees (of which the 2-3-4 tree is a special case) were not invented until 1972.

In many respects, the operation of 2-3 trees resembles that of 2-3-4 trees. Nodes can hold one or two data items and can have zero, two, or three children. Otherwise, the arrangement of the key values of the parent and its children is the same. The process of inserting a data item into a node is potentially simplified because fewer comparisons and moves are potentially necessary. As in 2-3-4 trees, all insertions are made into leaf nodes, and all leaf nodes are on the same level, as in the sample 2-3 tree shown in Figure 9-15.

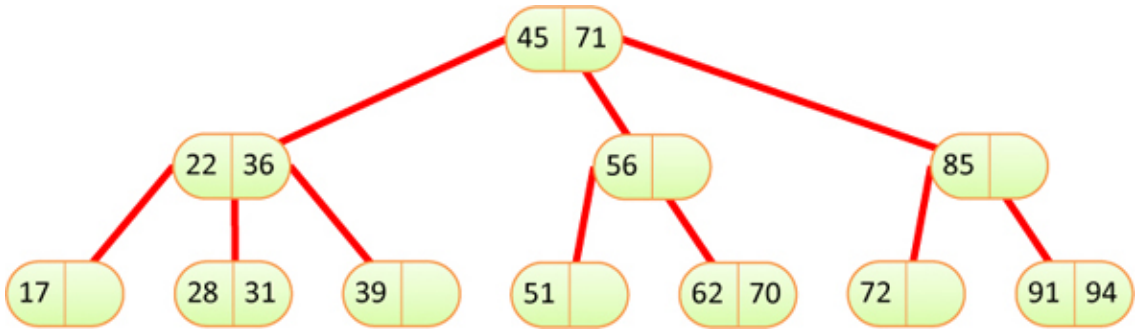

Figure 9-15 *A 2-3 tree*

Node Splits

You search for an existing data item in a 2-3 tree just as you do in a 2-3-4 tree, except for the number of data items and children. You might guess that insertion is also like that of a 2-3-4 tree, but there is a surprising difference in the way splits are handled.

Here's why the splits are so different. In either kind of tree, a node split requires three data items: one to be kept in the node being split, one to move right into the new node, and one to move up to the parent node. A full node in a 2-3-4 tree has three data items, which are moved to these three destinations. A full node in a 2-3 tree, however, has only two data items. Where can you get a third item? You must use the new item—the one being inserted in the tree.

In a 2-3-4 tree the new item is inserted after all the splits have taken place. In the 2-3 tree it must participate in the split. It must be inserted in a leaf, so no splits are possible on the way down. If the leaf node where the new item should be inserted is not full, the new item can be inserted immediately, but if the leaf node is full, it must be split. Its two items and the new item are distributed among these three nodes: the existing node, the new node, and the parent node. The three cases for how the items are distributed are shown in Figure 9-16.

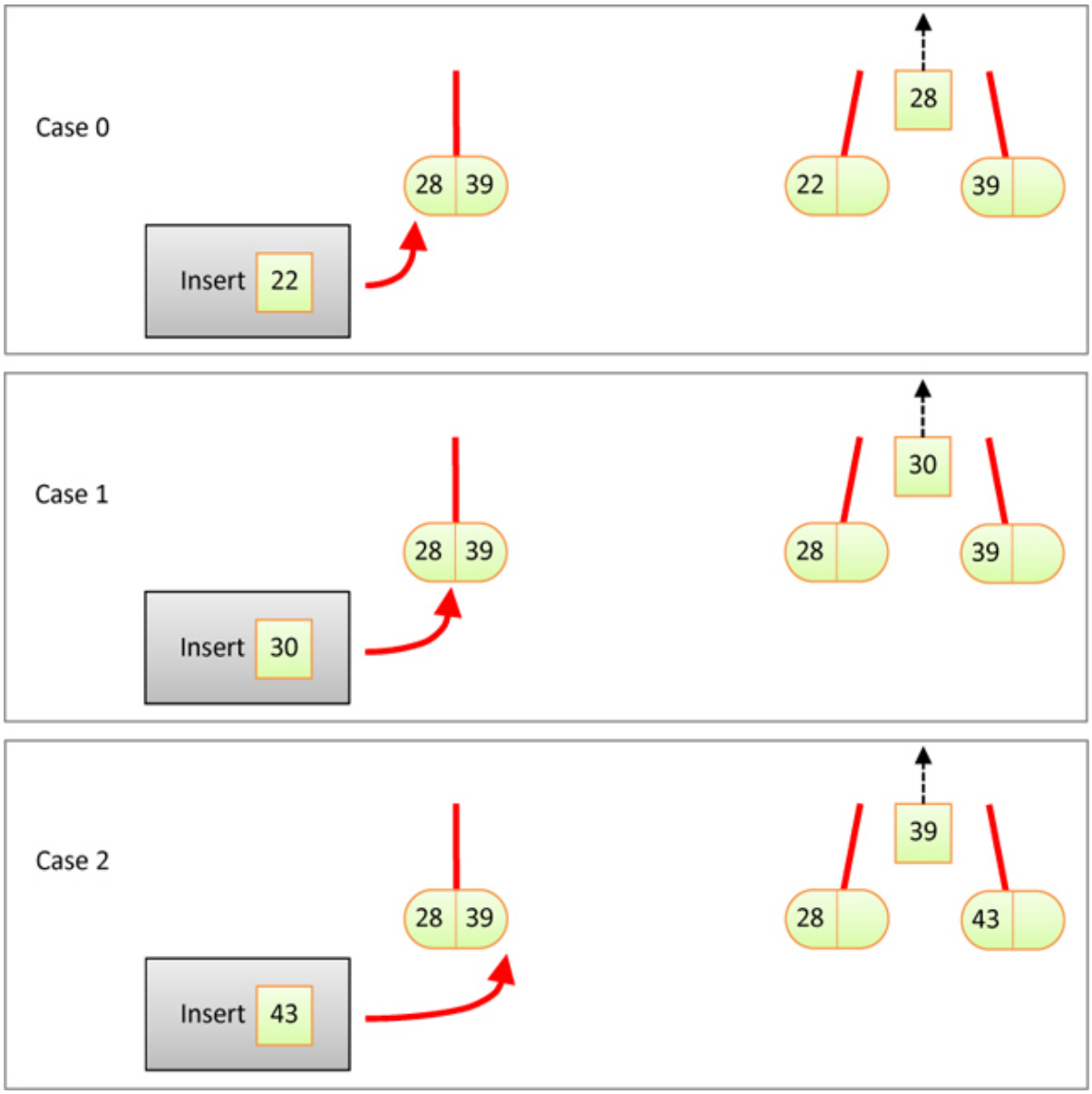

Figure 9-16 *Insertion into leaf nodes that are full causing splits*

In cases 0 and 2 the new item to insert goes into a leaf holding a single item. In case 1, when the new key falls between the two items that were in the leaf node being split, it must be passed to the parent node for insertion there. In all the cases, one item gets passed (or "promoted") to the parent node, as shown in the right part of the figure with the dashed black arrow pointing up. If the parent node has only a single item, the promoted item can be inserted in the node, a new child link attached, and the insertion is complete. If the parent node is full, the process must repeat, possibly continuing all the way up to the root node.

For example, inserting an item with key 47 in the 2-3 tree of Figure 9-15 would add the item to leaf node 51, moving item 51 over to the right to keep the items in the node ordered. Because the leaf node could accept the new item, nothing is promoted to its parent.

If you try inserting item 67 into the same tree, you find leaf node 62-70 to be full. This is an example of Case 1 because the new item's key is larger than 1 of the keys already stored there. Items 62 and 70 would be separated and item 67 would be promoted to the parent, node 56. Because the parent is not full, it can accept item 67 and the insertion is finished. Item 70 would be in the new split node while item 62 would remain in the leaf node that was split.

Promoting Splits to Internal Nodes

If you tried to insert a new item 27 in the 2-3 tree of Figure 9-15, the leaf node 28-31 would split and promote item 28 to its parent. That node 22-36 would split as well, and item 28 would be promoted again to its parent, the root node 45-71. The root node would also split, putting item 45 in the new root and making all paths to leaf nodes one level longer.

As you can see, the effects of promoting items up through internal nodes are complex. There's the item being promoted, and there's also the child links that need to be rearranged to keep the tree structure intact. The split cases shown in Figure 9-16 all show one item being promoted with two child links to the parent. The left-hand child link is the one that came from the parent when searching down for the leaf node to split. The right-hand link leads to the new node that was created by the split. It always contains a key that is larger than the key being promoted. That enables the parent node to insert the promoted item as item j and the promoted right-hand child link as child j+1. The original link to the node being split remains just to the left of the promoted item at child j.

Handling the child links gets a bit more confusing as the splits and promotions ripple up the tree. Figure 9-17 shows the cases that can occur when promotion finds an internal node that is already full. In each case, one item and one subtree are being promoted.

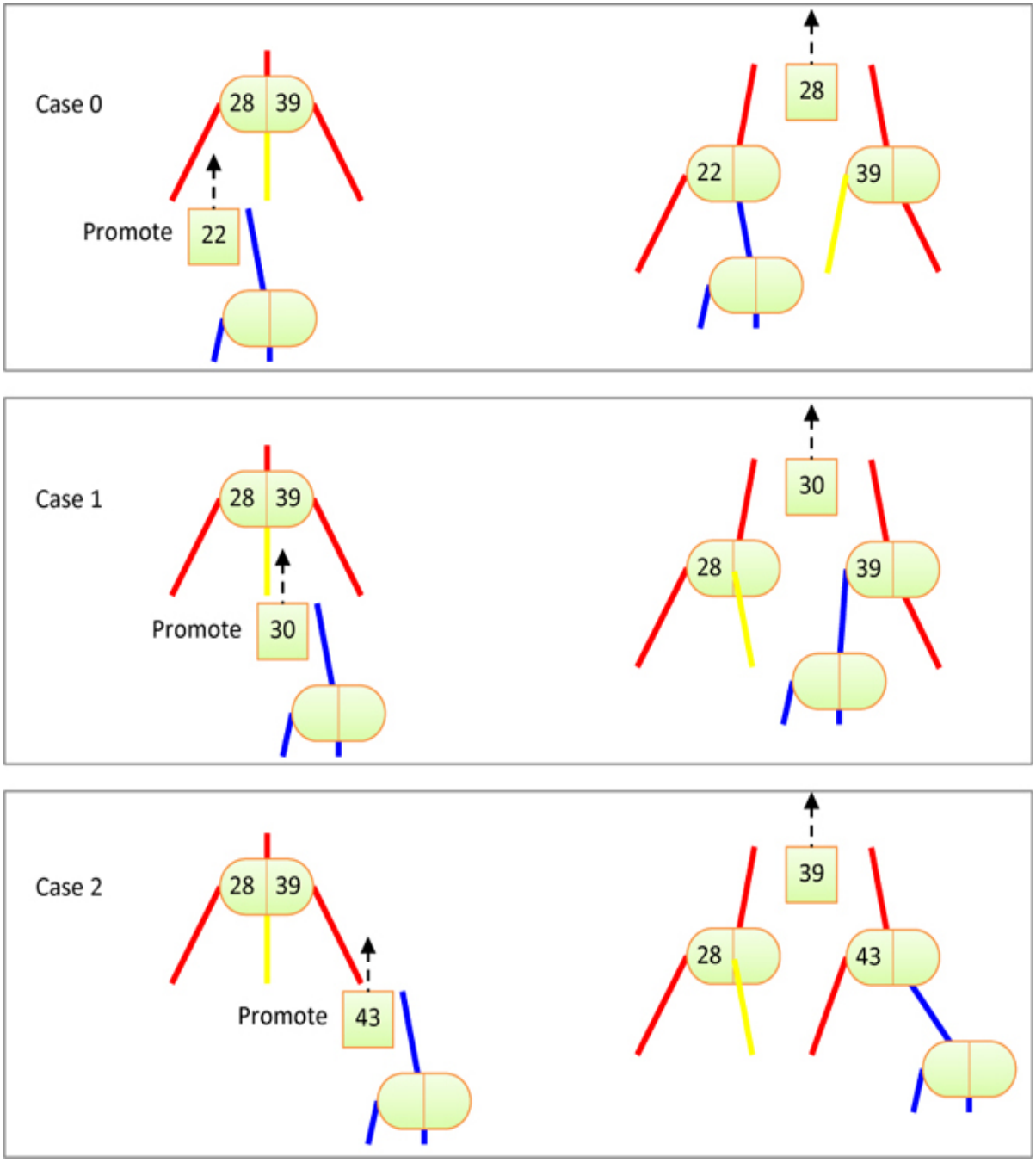

Figure 9-17 *Splitting internal nodes*

Just as for the leaf nodes, there are three cases, depending on which child link is the source of the split. What's tricky is that the child links and items all need different handling in the different cases. The examples in Figure 9-17 all show items 28 and 39 being split into different nodes but with different arrangements. Let's look at each case.

In Case 0, the split came from child 0, the lowest subtree. The promoted key, 22, must be lower than the lowest key of the node being split. To maintain the key order, the promoted key must replace key 0, and that key, 28, will be promoted next to its parent. After replacing key 0, you also must replace child 1 of the node being split with the right subtree that was being promoted from below, the blue link in Figure 9-17. The blue link could lead to a simple leaf node or a large subtree, depending on the split operation that happened below. All that this part of the algorithm needs to know, however, is that the blue link belongs to the right of the item that just replaced item 0.

What about the old values of child links 1 and 2 of the node being split? Child link 1 of the node being split is shown in yellow in Figure 9-17. In case 0, child 1's keys are above that of the item being promoted (28) and below that of the item moved to the new split node 39. Thus, the only place that child link 1 (yellow) can go is as child 0 of the new split node. Child link 2 of the node being split goes to child link 1 of the new split node.

In case 1, the promoted item and subtree come from child link 1 of the node about to be split. In this case the promoted item 30 continues to be promoted to the next parent. The promoted (blue) subtree, however, doesn't follow it up the tree. Instead, it becomes child 0 of the new split node because its keys are larger than item 30 (going up to the next parent) and less than the key of the new split node, 39. Child links 0 and 1 of the node being split remain as before, and child 2 moves over to become child 1 of the new split node because it holds all the keys above 39.

Case 2 is perhaps the easiest of the three. The promoted item key, 43, goes into the new split node, and the promoted (blue) subtree becomes child 1 of that split node. Item 39 gets promoted from the node being split to the next parent along with a link to the new split node 43. Child 2 of the node being split moves over to become child 0 of the new split node because all its keys are greater than 39 and less than 43.

Implementation

We leave a complete Python implementation of insertion in a 2-3 tree as an exercise. We finish with some hints on how to handle splits.

On the way down, the insertion routine doesn't notice whether the nodes it encounters are full or not. It searches down through the tree until it finds the

appropriate leaf. If the leaf is not full, it inserts the new value and the insertion is done. If the leaf is full, however, the routine must rearrange the tree to make room. To do this, there are a couple of options. It could call a `split()` method. Parameters to this method would include the full leaf node and the new item to insert. It would be the responsibility of `split()` to make the split and insert the new item in the new leaf, and then promote an item to the parent.

If `split()` finds that the leaf's parent is full, it would call itself recursively to split the parent. It keeps calling itself until a non-full node or the root is found. The return value of `split()` is the new right node, which can be used by the previous incarnation of `split()`.

How would `split()` know the parent node? If `split()` is called on a leaf node along with an item to insert, it doesn't have a direct link to the leaf node's parent, at least not if you follow the tree and node structure that we used for the 2-3-4 trees. There are several ways to address that. First, you could add a `parent` field to each `Node` object to have an explicit link in every node. The root node would have `None` or possibly the 2-3 tree object for its `parent` value. When creating nodes, the constructor would need a `parent` parameter. Then when you rearrange the child links while promoting items from splits, you would rearrange the parent links at the same time. This is about the same amount of effort needed to make doubly linked lists from singly linked ones, as discussed in Chapter 5.

The second option would be to keep a stack of node objects that represents the path to the node being split. The bottom of the stack would be the root node of the tree, and the top would initially be the leaf node to split. This stack could replace the node parameter to `split()`, and finding the parent would mean popping the top item off the stack. This process is fairly straightforward, although the `insert()` method would need to build the stack, or you could modify the `__find()` method to return it as a secondary return value.

The third option is to use the recursion that is naturally part of the `__find()` process in the `insert()` implementation. Each call to `insert()` would try to insert the new item if the node is a leaf and return an item to promote if the node was full. In other words, as `insert()` recursively calls itself on child nodes, it would check the return values to see whether it was done (no promoted item requiring more splits), or whether there is an item to promote that must be inserted on the current node. If a promoted item is received from a recursive call and the node is not full, then the promoted item can be inserted, and no further splits or promotions are needed above that node in the tree. If

the node is full, it makes the new split node, rearranges the items and child links, and returns the item to promote according to the cases described in Figure 9-17. The recursive changes ripple back up to the root, and if the root is full, it would be split too.

Coding the splitting process using any of these options is complicated by the case logic for the promoted items and their accompanying subtrees. We've mentioned the promoted item as a parameter for the `split()` method or as a return value for `insert()` method. There is both the key and data value for that item as well as the accompanying subtree that must be passed or returned.

Efficiency of 2-3 Trees

We haven't shown the deletion operation for 2-3 trees, but as you might imagine, it involves rotation and fusion operations like what was done for 2-3-4 trees. That means that 2-3 trees have largely the same efficiency as 2-3-4 trees. Search time is proportional to the height of the tree. If every node is full, each internal node has three children, and the height is $\log_3(N)$, where N is the number of items. With the same analysis as for 2-3-4 trees, you end up with a search being an O(log N) operation.

Insertion also descends the height of the tree but sometimes must go back up the path to the root, splitting full nodes to make room for the inserted item. That means insertion takes somewhere between $\log_3(N)$ and $2\times \log_3(N)$ steps. That's less efficient than the 2-3-4 tree, but only in terms of a constant multiplier. Overall both are still O(log N) operations.

In terms of memory usage, because it stores at most two items and three child links at each node, the amount of unused memory will be less for a 2-3 tree than for a 2-3-4 tree.

External Storage

Remember that 2-3-4 trees are examples of multiway trees, which can have more than two children and more than one data item. Another kind of multiway tree, the B-tree, is useful when data resides in external storage. **External storage** typically refers to some kind of disk system containing **files**, such as the hard disk found in many desktop computers and servers. For cloud computing, chunks of data are stored together and can be part of a **dataset**